Assignment No. 7

Name : Ashutosh Sanjay More **PRN :** 124B1F036

Batch: A2 **Div :** A **Course :** Design and Analysis of Algorithms

Github Link : https://github.com/Ashutoshmore24/DAA_Submission.git

---

University Timetable Scheduling Using Graph Colouring

1. Problem Statement

A university faces challenges in scheduling examination timetables because students may be enrolled in multiple courses. If examinations for courses taken by the same student are scheduled simultaneously, a timetable conflict occurs.

The objective is to develop an efficient examination scheduling system using the Graph Colouring technique. Each course is represented as a vertex in a graph, and an edge connects two vertices when the corresponding courses share at least one student. Each vertex is assigned a colour representing an examination time slot, ensuring that no two adjacent vertices receive the same colour.

The system should minimize the number of examination slots while maintaining conflict-free scheduling. It should also allocate suitable examination rooms according to student enrolment and room capacity.

2. Aim

To design and implement an examination timetable scheduling system using Graph Colouring, ensuring that courses with common students are assigned different examination slots while minimizing the number of slots and allocating appropriate examination rooms.

3. Course Objectives

1. To understand the computational complexity of various algorithms.
2. To select appropriate algorithm design strategies for solving real-world problems.

4. Course Outcomes

1. Apply Backtracking strategies to solve various problems.
2. Classify computational problems into P, NP, NP-complete, and NP-hard categories.

5. Theory

5.1 Graph Theory

Graph Theory is a mathematical framework used to represent objects and the relationships between them.

A graph is represented as G = (V, E), where:

- V represents the set of vertices.
- E represents the set of edges connecting pairs of vertices.

Graphs can be directed or undirected and weighted or unweighted.

In university examination scheduling, the graph is undirected and unweighted because a scheduling conflict between two courses is mutual and does not require an edge weight.

Each vertex represents a course, and an edge represents a conflict caused by one or more students being enrolled in both courses.

5.2 Graph Colouring

Graph Colouring is the process of assigning colours to the vertices of a graph such that no two adjacent vertices share the same colour.

In examination scheduling:

- Each vertex represents a course.
- Each edge represents a conflict between two courses.
- Each colour represents an examination time slot.

If two courses are connected by an edge, they must be assigned different examination slots.

The objective is to minimize the number of colours required while satisfying all conflict constraints.

The minimum number of colours required to colour a graph properly is called its **chromatic number**, denoted by $\chi(G)$.

Finding the chromatic number of an arbitrary graph is an NP-hard optimization problem. Greedy colouring algorithms provide practical solutions but do not always guarantee the minimum number of colours.

5.3 Conflict Graph Modelling

The examination scheduling problem can be represented using a conflict graph.

The graph is constructed as follows:

1. Create one vertex for every course.
2. Examine the courses taken by each student.

3. Add an edge between every pair of distinct courses taken by the same student.
4. Ensure that each conflict edge is represented only once.
5. Assign colours to the vertices using a graph colouring algorithm.

For example, if Student S1 is enrolled in courses C1 and C2, an edge is created between C1 and C2.

If another student is enrolled in C2 and C3, an edge is created between C2 and C3.

The resulting graph represents all course conflicts. A valid colouring ensures that students do not have overlapping examinations.

5.4 Greedy Graph Colouring

The Greedy Graph Colouring algorithm assigns colours to vertices one at a time.

For each vertex, it selects the smallest available colour that has not been assigned to any of its already-coloured adjacent vertices.

Working:

1. Initialize all vertices as uncoloured.
2. Select a vertex according to the chosen ordering.
3. Identify the colours already assigned to its adjacent vertices.
4. Assign the smallest available colour.
5. Repeat until all vertices are coloured.

Advantages:

- Simple to implement.
- Efficient for many practical graphs.
- Suitable for large datasets when implemented with adjacency lists.

Limitation:

The result depends on the vertex ordering. Greedy colouring does not guarantee the minimum number of colours.

A straightforward implementation may take $O(V^2)$ time, where V is the number of vertices.

5.5 Welsh-Powell Algorithm

Welsh-Powell is a graph colouring heuristic that improves the vertex ordering used by Greedy Colouring.

It sorts the vertices in descending order of their degrees, where the degree of a vertex is the number of adjacent vertices.

Working:

1. Calculate the degree of every vertex.
2. Sort the vertices in descending order of degree.
3. Select the first uncoloured vertex and assign the current colour.
4. Assign the same colour to other uncoloured vertices that are not adjacent to any vertex already assigned that colour.
5. Repeat with a new colour until every vertex is coloured.

Advantages:

- Prioritizes highly connected courses.
- Often produces better colourings than an arbitrary vertex ordering.
- Is relatively straightforward to implement.

Limitation:

Welsh-Powell is a heuristic and does not guarantee the chromatic number of the graph.

Its complexity depends on the implementation. With adjacency lists, sorting takes $O(V \log V)$, while a straightforward colouring implementation may take $O(V^2)$ time.

5.6 DSATUR Algorithm

DSATUR stands for Degree of Saturation. It selects the next vertex according to the number of distinct colours already assigned to its adjacent vertices.

The saturation degree of an uncoloured vertex is the number of different colours present among its coloured neighbours.

Working:

1. Calculate the degree of each vertex.
2. Select an uncoloured vertex with the highest saturation degree.
3. If multiple vertices have the same saturation degree, select the one with the highest degree.
4. Assign the smallest available colour.
5. Update the saturation degrees of neighbouring vertices.
6. Continue until all vertices are coloured.

Advantages:

- Adapts its vertex selection to the colouring already performed.
- Often produces effective colourings for complex conflict graphs.
- Can also be used within exact graph colouring methods.

Limitation:

The basic DSATUR heuristic does not guarantee an optimal colouring. A straightforward implementation can take $O(V^2)$ time, although optimized implementations may perform better on suitable graphs.

5.7 Comparison of Graph Colouring Algorithms

Feature	Greedy Colouring	Welsh-Powell	DSATUR
Vertex selection	Follows a chosen order	Descending degree order	Highest saturation degree
Main strategy	Assign the smallest available colour	Colour highly connected vertices first	Dynamically select the most constrained vertex
Implementation	Simple	Simple to moderate	Moderate
Typical performance	Depends on vertex order	Often improves on arbitrary ordering	Often effective for complex conflict graphs
Guarantees minimum colours	No	No	No, for the basic heuristic
Typical basic implementation complexity	$O(V^2)$	$O(V^2)$	$O(V^2)$
Best use	Simple scheduling	Degree-based scheduling	More challenging conflict graphs

The actual performance of these algorithms depends on the graph's structure and the implementation. Their results should be compared using the number of colours produced and execution time on the same conflict graph.

5.8 Room Allocation Constraints

After examination slots have been assigned, each examination must be allocated a suitable classroom.

Room allocation must satisfy the following conditions:

1. **Capacity:** The room must accommodate all students registered for the examination.
2. **Availability:** A room can host only one examination during a given time slot.
3. **Conflict-free scheduling:** Courses sharing students must not be scheduled simultaneously.
4. **Space utilization:** A suitable room should be selected without unnecessarily using a larger room when a smaller one is available.

A practical room-allocation strategy is to process examinations within each slot in descending order of student enrolment and select the smallest available room with sufficient capacity.

This is a heuristic for room assignment. If no suitable room is available, the system should report the allocation failure rather than assign an unsuitable room.

5.9 Optimization for Large Datasets

A university may have thousands of students and courses. The following techniques can improve performance:

- **Adjacency Lists:** Store only existing conflict edges instead of using a full adjacency matrix.
- **Hash Maps and Sets:** Store student-course relationships and avoid duplicate conflict edges.
- **Degree-Based Ordering:** Prioritize courses with many conflicts.
- **Efficient Traversal:** Process only neighbouring vertices when determining available colours.
- **Optimized Data Structures:** Use suitable priority queues and saturation tracking for larger DSATUR implementations.
- **Room Allocation:** Maintain room availability separately for each time slot.

If a student is enrolled in k courses, constructing conflicts for that student may generate up to $k(k-1)/2$ course pairs. Therefore, the cost of constructing the conflict graph depends on the enrolment distribution as well as the number of students and courses.

6. Algorithm

Algorithm: University Timetable Scheduling Using Welsh-Powell Colouring

Input:

- A list of students and their enrolled courses.
- A list of available rooms and their capacities.

Output:

- A conflict-free timetable containing the course, examination slot, and assigned room.

1. Create a vertex for every course.
2. For each student, create an edge between every pair of distinct courses taken by that student.
3. Store the graph using adjacency lists.
4. Calculate the degree of every vertex.
5. Sort vertices in descending order of degree.
6. Initialize all vertices as uncoloured.
7. Process vertices in sorted order:
    - Identify the colours assigned to adjacent vertices.
    - Assign the smallest available colour.
8. Determine the number of examination slots used.
9. For each slot, sort its examinations by descending student enrolment.
10. Assign each examination the smallest available room that has sufficient capacity.
11. Report any examination for which no suitable room is available.
12. Display the final timetable and the total number of slots used.

7. Pseudocode

```
UNIVERSITY-TIMETABLE(Courses, Students, Rooms)

    Create an empty adjacency list for every course

    For each student:
        Get the distinct courses taken by the student

        For each pair of courses (c1, c2):
            Add an edge between c1 and c2
            Add an edge between c2 and c1

    Sort courses by descending degree

    Initialize all course colours as unassigned

    For each course c in sorted order:
        Find the colours used by adjacent courses
        Assign the smallest available colour to c

    Determine the total number of colours used

    For each examination slot:
        Find all courses assigned to that slot
        Sort them by descending student enrolment

        For each course:
            Find the smallest available room
            whose capacity is sufficient

            If a suitable room exists:
                Assign the room to the course
                Mark the room as occupied for this slot
            Else:
                Report room allocation failure

    Display each course, its slot, and its room
```

8. Example Input

Student Enrolments

Student	Enrolled Courses
S1	C1, C2
S2	C2, C3
S3	C1, C3
S4	C4

Available Rooms

Room	Capacity
R1	100
R2	80
R3	50

The example contains four courses and three rooms.

Courses C1, C2, and C3 form a triangle in the conflict graph because each pair shares at least one student. Consequently, these three courses require different examination slots.

Course C4 has no conflicts and can share a slot with any of the other courses, provided room availability is respected.

9. Expected Scheduling Results

The Welsh-Powell algorithm assigns three different colours to C1, C2, and C3. Course C4 can use the first slot because it has no adjacent vertices.

Assuming the room-allocation policy selects the smallest available room that fits each examination, the resulting timetable is:

Course	Time Slot	Number of Students	Room
C1	Slot 1	2	R3
C2	Slot 2	2	R3
C3	Slot 3	2	R3

C4	Slot 1	1	R2

Room R3 has a capacity of 50, which is sufficient for each examination. Since C1 and C4 occur in the same slot, they are assigned different rooms.

Rooms may be reused in different slots because they are not occupied by the same examination throughout the entire timetable.

Explanation:

1. C1, C2, and C3 require separate slots because every pair conflicts.
2. C4 can share Slot 1 with C1 because it has no conflict with C1.
3. C1 and C4 are assigned different rooms because both examinations occur in Slot 1.
4. The assigned rooms have sufficient capacity for all students.
5. The timetable uses three examination slots.

10. Time and Space Complexity

Let:

- V = Number of courses.
- E = Number of unique conflict edges.
- S = Number of students.
- k_i = Number of distinct courses taken by student i.

Conflict Graph Construction

The time required to construct the conflict graph is proportional to the number of student-course enrolments plus the course pairs generated from each student's enrolments:

$O(S + \Sigma k_i^2)$

This expression assumes that the enrolment lists have already been read and that adjacency sets provide efficient duplicate handling. The exact cost also depends on the data structures used.

Welsh-Powell Colouring

- Sorting vertices: O(V log V).
- Colour assignment: $O(V^2)$ in a straightforward implementation.
- Total colouring time: $O(V^2)$ in the worst case for the implementation used here.
- Graph storage: O(V + E).
- Colour and degree arrays: O(V).

Room Allocation

The time required depends on the number of examinations, the number of rooms, and the sorting strategy. A straightforward implementation that sorts examinations and rooms and checks rooms sequentially can take up to $O(V \log V + V R \log R + V R)$, where R is the number of rooms.

Overall Space Complexity

The conflict graph requires $O(V + E)$ space, with additional storage for colours, enrolment counts, room assignments, and room availability.

11. Conclusion

The University Timetable Scheduling problem demonstrates the practical application of Graph Colouring to examination scheduling.

Courses are represented as vertices, and shared student enrolments are represented as conflict edges. The Welsh-Powell algorithm assigns examination slots by prioritizing courses with higher degrees, helping to produce a conflict-free timetable.

Room allocation extends the solution by ensuring that each examination has a suitable classroom and that no room is assigned to two examinations during the same slot.

Although Welsh-Powell does not guarantee the minimum possible number of slots, it provides a practical heuristic for scheduling. More advanced approaches, such as DSATUR or exact Backtracking, can be considered when improved colouring quality is required.

Thus, Graph Colouring provides an effective foundation for developing a conflict-free and resource-aware university examination scheduling system.
